University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 2, 2023

Answer all questions.
Approximate marks per question are shown in brackets.

Time: 40 minutes **Marks: 35**

Section 1. ASD2: Software Architecture [10 Marks]

Question 1 [4 Marks]

Three views in Krutchen's "4+1" views of architecture are the Use Case/Scenario View, the Development/Allocation View, and the Physical/Deployment View. Name the other 2 views and briefly describe them (1 sentence each).

1 mark for each name, 1 mark for each description.

Logical or Module View: (can give either name)
EITHER concerned about the output(s) of the system and how it will affect the end users;
OR splits the system into a set of abstractions, or modules.

Process or Component-and-Connector View: (can give either name)
EITHER deals with concurrency and distribution, system integrity, and fault tolerance.
OR explains which components interact, and how they do so
OR describes the dynamic connections between different components at runtime.

Question 2 [2 Marks]

Why are 2 architecture patterns common in language processing systems, rather than one?

1 mark for each pattern-and-its-usage

If they have a good reason for any other pattern appropriately used for lang processing, accept that too

pipe and filter is used when there is no user interaction, because then the steps of compiling a program happen strictly in a set sequence (however the put this point across is fine)

repository is used when a programming IDE is used the separate components do not run strictly in sequence so that the programmer can be made aware of errors immediately, can have autocompletion, etc. (any one such example of respository for IDE usage)

Question 3 [4 Marks]

To help us identify services in SOA, we consider 2 types: task-oriented and entity-oriented. In one sentence each, give an example of each type in the context of software for a university, stating briefly why it is an example of that type.

1 mark for each valid example, 1 mark for each reason showing they know what it is:

task-oriented: relating to any uni function, e.g. application, registration, graduation, etc.

entity-oriented: relating to any object, typically a form like a registration form etc.

Section 2. ToA: Theory of Algorithms [16 Marks]

Question 4 [3Marks]

You have been given five variants of an algorithm for achieving the same task. You have analysed the number of times the basic operation is executed based on the input and determined that it can described as follows, for each variant:

Variant 1: n^2
Variant 2: 6n
Variant 3: n!
Variant 4: n log n
Variant 5: 2^n

Order the algorithm variants from best to worse efficiency --- for large input sizes.

In order:

6n
nlogn
n^2
2^n
n!

3 marks if completely correct, 2 marks if any 1 error, 1 mark if any 1 correct

Question 5 [3 Marks]

Given the multiplication algorithm (à la Russe) below:

```
ALGORITHM multiply(n, m)
   t  = 0
   while n != 1 do
         if n % 2 != 0 do
               t += m
         m = m * 2
         n = n // 2
   t += m
   return t
```

a. To which algorithmic pattern/strategy, among the following, does the algorithm belong? [1]

(a) divide and conquer
(b) decrease by a constant
(c) **decrease by a constant factor**
(d) brute force
(e) transform and conquer

b. Does the algorithm exhibit the same performance, from a time perspective, when the values of n and m are swapped (e.g., n=3 and m=10 vs. n = 10 and m = 3)? Explain. [2]

No **[1]**. The number of executed steps is determined by how many times n must be halved until it reaches 1 (i.e., problem size is characterised by n). As such, a larger value of n implies a larger number of execution steps. For instance, consider the case where n=3 and m=10. When n=3, a single loop is sufficient whereas three are necessary for n=10 **[1]**

Question 6 [10 Marks]

Given the algorithm below:

```
ALGORITHM func(A, l, r, i):
  if l < r:
        v = (l + r) // 2
        if A[v] % i == 0 do
              PRINT v
        else:
              func(A, v + 1, r, i)
              func(A, l, v - 1, i)
```

a. What does the algorithm do? [2]

Uses divide and conquer to search and print some element(s) from A that are divisible by i. The values must be at the centre of each segment of A.

b. Write pseudo-code for the brute-force version of the algorithm that prints all the values that satisfy line 4's condition? [2]

```
(let n be the size of A)
for idx in [0 ... n] do
   if i%A[idx] == 0 do
         PRINT i
```

c. Use the master theorem to show that both the algorithm as presented and its expanded brute-force version belong to the same efficiency class in the worst case (Show your work).
[6]

Solve using the master theorem

If $T(n) = aT(n/b) + f(n)$ and $f(n) \in \Theta(n^d)$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$

In the case of the brute-force solution, worst case performance is seen when the element is at the last position (n-1) **[1].** We can compute the sum from 0 to n-1 of doing a single comparison 1 + 1 + …. + 1 = n hence T(n) in BigO(n) **[1]**

In the case of the divide and conquer approach, we have the recurrence relations T(0) = 0 and T(n) = aT(n/b) + f(n) where a=2**[1],** b=2**[1]** and f(n) in BigTheta(n^0) so d = 0 **[1].** T(n) in BigTheta(n^log_2^2) = BigTheta(n). Hence T(n) in BigO(n) --- by definition of BigTheta **[1].**

Section 3. ToC: Theory of Computation [9 Marks]

Question 9 [1 Marks]

Consider the following Turing machine with states q and p, where q is the start state, Γ = {0, 1, □} (where the box "□" indicates the blank symbol), Σ = {0,1}, and the following transition functions:

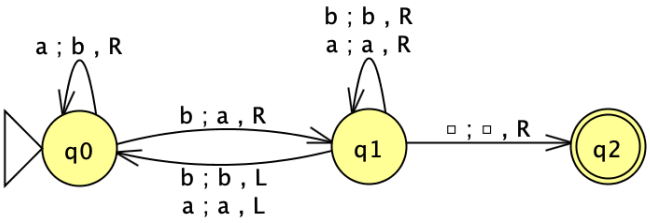
State	Tape Symbol	Move
q	0	(q,0,R)
q	1	(p,0,R)
q	□	(q, □,R)
p	0	(q,0,L)
p	1	none (halt)
p	□	(q,0,L)

Describe a property of the input strings that makes this TM halt. Then select the string from the options listed which make this TM halt.

a. 10010
b. 0100
c. 0001
d. 10110

Question 10 [2 Marks]

Consider the following state diagram of a Turing machine, with q_0 the start state and q_2 the final state, and where the □ (small square box) denotes blank.



The TM with these states and transitions is given input w = aaababa. Examine the TM on this input and then select the correct answer from the following options:

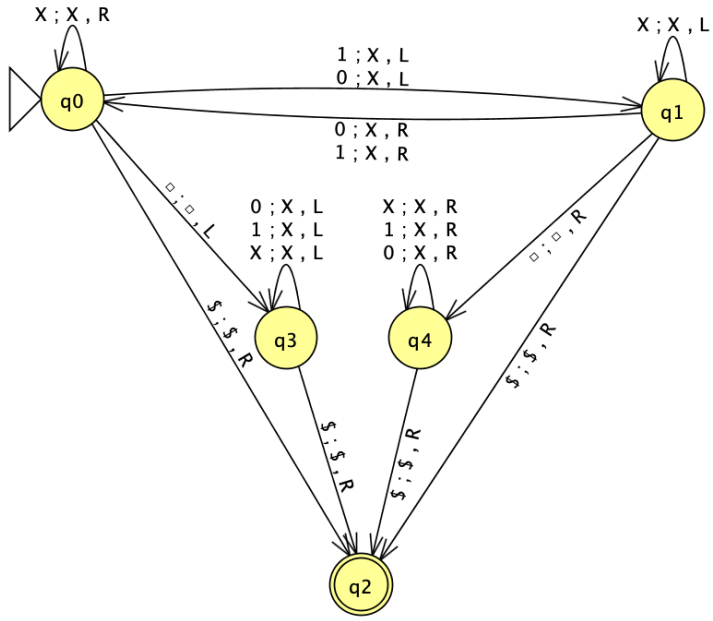
a. Reachable IDs include bbbaaaq$_1$a and bbbaaaq$_1$b.
b. Reachable IDs include bbbaaq$_1$ba and bbbaaba□q$_2$□.

Question 11 [6 Marks]

Design a deterministic Turing Machine such that, when its head is on some arbitrary position somewhere on the input that is of pattern w$w with w from the alphabet {0,1}, it will find the centre symbol $ and accept. So, it should accept, e.g., 0001$0001 and 1$1 on the tape, but reject 11.

Hint: first describe in short statements the three component tasks that the TM needs to accomplish. Then draw the state diagram that accomplishes that.

Answer: alike the one from the exercises with the blanks and using #, but then with marking off 0 and 1 with an X. Marking note: blank can be indicated with a □, B or a b with a diagonal line through it.



Marking guide: **deduct 0.5** for not indicating start state (an incoming arrow, -->), and **deduct 0.5** for not indicating final state (double lined circle). **A state diagram as above gains the 6 marks.**

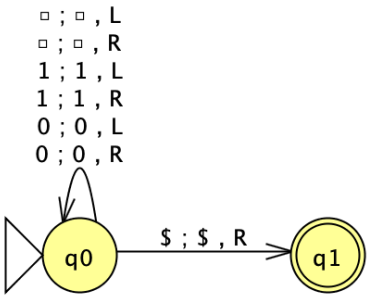
Give **2 marks** if they can describe how their TM would solve it even if their state diagram is wrong or is not present. It should say something along the line of:

1) it alternates looking right and left (*marking note*: or vv) for $, marking off each cell it has seen;
2) when it encounters the blank, it will go only in the other direction;
3) when it encounters $, it goes to the final state.

With **0.5** for each sub-task and **another 0.5** for having all three.

If they write all the transition functions as functions, then also give the full marks.

If they draw an NTM that is correct ('guess whether $ is to the left or to the right' + 'goto final upon $'), give **1 mark**. The 'simplest' NTM is:



-END-